Diagrammi completi

Indice

I diagrammi usano Mermaid e possono essere visualizzati in GitHub o in Mermaid Live Editor.

1. Casi d’uso

```
flowchart LR
  Player[Giocatore]
  Local[Admin locale]
  GameAdmin[Admin gioco]
  Platform[Admin piattaforma]

  Player --> U1[Vedere giochi]
  Player --> U2[Vedere proprie partite]
  Player --> U3[Vedere statistiche]
  Player --> U4[Vedere tornei]

  Local --> U5[Gestire giochi del locale]
  Local --> U6[Gestire edge e sensori]
  Local --> U7[Gestire attuatori]
  Local --> U8[Creare squadre]
  Local --> U9[Avviare partite]
  Local --> U10[Vedere statistiche locali]

  GameAdmin --> U11[Definire tipi di gioco]
  GameAdmin --> U12[Definire regole eventi]
  GameAdmin --> U13[Definire modelli sensore]

  Platform --> U14[Gestire locali]
  Platform --> U15[Gestire utenti]
  Platform --> U16[Creare tornei]
  Platform --> U17[Scegliere locali e squadre]
  Platform --> U18[Vedere statistiche globali]
```

2. Diagramma del dominio

```
classDiagram
  Locale "1" --> "*" User
  Locale "1" --> "*" Game
  Locale "1" --> "*" EdgeDevice
  Locale "1" --> "*" Team
  GameType "1" --> "*" Game
  GameType "1" --> "*" SensorTemplate
  EdgeDevice "1" --> "*" Sensor
  EdgeDevice "1" --> "*" Actuator
  Game "1" --> "*" Sensor
  Game "1" --> "*" Actuator
  Game "1" --> "*" Match
  Match "1" --> "*" MatchEvent
  Team "1" --> "*" TeamMember
  User "1" --> "*" TeamMember
  Tournament "*" --> "*" Locale : tournament_locations
  Tournament "*" --> "*" Team : tournament_teams
  Tournament "1" --> "*" TournamentMatch
  TournamentMatch "*" --> "1" Match

  class User { id username password role locale_id }
  class GameType { id name description score_limit supports_teams }
  class SensorTemplate { id name event_type description }
  class Game { id name type status locale_id game_type_id }
  class EdgeDevice { id name status last_seen last_sync }
  class Sensor { id name sensor_type mqtt_topic status }
  class Actuator { id name actuator_type state mqtt_topic }
  class Match { id participant_mode player1_name player2_name score1 score2 status }
  class MatchEvent { id event_uuid event_type sync_status created_at }
  class Tournament { id name game_type participant_mode status }
  class TournamentMatch { round_number scheduled_at }
  class Team { id name locale_id }
```

3. Diagramma dei package

```
flowchart TB
  subgraph Frontend
    HTML[Pagine HTML]
    JS[JavaScript browser]
    CSS[CSS]
  end

  subgraph Gateway
    G[server.js - instradamento]
  end

  subgraph Backend
    R[Routes]
    M[Middleware]
    C[Controllers]
    S[Services]
    U[Utils/Rules]
    DB[db.js]
  end

  subgraph Edge
    EAPI[Express API locale]
    QUEUE[Coda JSON]
    EMQTT[Client MQTT]
    EUI[Interfaccia locale]
  end

  HTML --> JS
```

```
  JS --> G
  G --> R
  R --> M
  M --> C
  C --> S
  C --> U
  S --> U
  C --> DB
  S --> DB
  EUI --> EAPI
  EAPI --> QUEUE
  EAPI --> EMQTT
```

4. Classi/moduli di implementazione

```
classDiagram
  class ApiGateway { serviceFor(path) forwardRequest() health() }
  class AuthController { login() }
  class UserController { getUsers() createClient() createLocalAdmin() createGameAdmin() }
  class GameTypeController { getGameTypes() createGameType() createSensorTemplate() }
  class DeviceController { getDevices() createSensor() createActuator() }
  class MatchController { startMatch() addMatchEvent() simulateMatchMqtt() }
  class MatchEventService { processMatchEvent() getMatchRow() getMatchEvents() }
  class ActuatorService { updateActuators() }
  class TournamentController { createTournament() addMatchToTournament() }
  class TournamentService { getTournamentMatches() getTournamentRankingRows() }
  class TournamentRules { calculateTournamentRanking() isTournamentMatchCompatible() }
  class ValidationRules { validateGameTypeInput() validateTournamentInput() validateMatchParticipants() }
  class EdgeService { publishOrQueue() syncQueue() simulate() }

  ApiGateway --> AuthController
  ApiGateway --> MatchController
  ApiGateway --> TournamentController
  MatchController --> MatchEventService
  MatchEventService --> ActuatorService
  MatchEventService --> TournamentService
  TournamentController --> TournamentService
  TournamentService --> TournamentRules
  GameTypeController --> ValidationRules
  MatchController --> ValidationRules
```

5. Microservizi con input, output e logica

```
flowchart LR
  Browser -->|REST /api| Gateway
  Gateway -->|auth users locales games game-types devices| Catalog[Catalog Service]
  Gateway -->|matches| Match[Match Service]
  Gateway -->|teams tournaments statistics| Tournament[Tournament Service]
  Edge -->|MQTT eventi sensori| Broker[Mosquitto]
  Broker --> Match
  Match -->|MQTT comandi attuatori| Broker
  Broker --> Edge
  Catalog --> MySQL[(MySQL)]
  Match --> MySQL
  Tournament --> MySQL

  C1[Catalog logic: ruoli, configurazione, inventario] -.-> Catalog
  M1[Match logic: punteggio, UUID, eventi] -.-> Match
  T1[Tournament logic: calendario, compatibilita, classifica] -.-> Tournament
  E1[Edge logic: sensori, coda offline, heartbeat] -.-> Edge
```

6. Sequenza login

```
sequenceDiagram
  actor U as Utente
  participant B as Browser
  participant G as API Gateway
  participant C as Catalog Service
  participant D as MySQL
  U->>B: username e password
  B->>G: POST /api/auth/login
  G->>C: inoltra richiesta
  C->>D: SELECT user
  D-->>C: utente e ruolo
  C-->>G: JSON utente
  G-->>B: risposta
  B-->>U: dashboard del ruolo
```

7. Sequenza partita MQTT

```
sequenceDiagram
  actor A as Admin locale
  participant F as Frontend
  participant G as Gateway
  participant M as Match Service
  participant E as Edge Service
  participant Q as MQTT Broker
  participant D as MySQL
  A->>F: Avvia partita
  F->>G: POST /api/matches/start
  G->>M: richiesta
  M->>D: INSERT match + UPDATE game
  M-->>F: match LIVE
  A->>F: Simula MQTT
  F->>G: POST /matches/id/simulate-mqtt
  G->>M: richiesta
  M->>E: POST /simulate
  loop eventi
    E->>Q: publish evento QoS 1
    Q->>M: evento MQTT
    M->>D: controllo UUID e UPDATE punteggio
    M->>Q: comando attuatore
    Q->>E: stato display/LED
  end
```

8. Sequenza offline e sincronizzazione

```
sequenceDiagram
  participant S as Sensore simulato
  participant E as Edge
  participant Q as Broker
  participant M as Match Service
  participant D as MySQL
  S->>E: evento
  alt MQTT offline
    E->>E: salva evento PENDING in JSON
  else MQTT online
    E->>Q: publish evento
  end
  E->>Q: riconnessione
  E->>E: legge coda
  E->>Q: ripubblica evento con stesso UUID
  Q->>M: evento
```

```
  M->>D: verifica UUID
  M->>D: salva una sola volta
  E->>E: svuota coda
```

9. Sequenza torneo

```
sequenceDiagram
  actor P as Admin piattaforma
  participant F as Frontend
  participant T as Tournament Service
  participant D as MySQL
  P->>F: crea torneo con locali e modalita
  F->>T: POST /api/tournaments
  T->>D: INSERT tournament
  T->>D: INSERT tournament_locations
  T->>D: INSERT tournament_teams
  P->>F: collega partita e turno
  F->>T: POST /tournaments/id/matches
  T->>D: verifica tipo, modalita, locale e stato
  T->>D: INSERT tournament_matches
  T->>D: SELECT partite concluse
  T-->>F: classifica calcolata
```

10. Deployment Docker

```
flowchart TB
  Host[Computer studente]
  Host --> Frontend[frontend :8080]
  Host --> Gateway[api-gateway :3000]
  Host --> Edge[edge-service :8090]
  Gateway --> Catalog[catalog-service :3001 interno]
  Gateway --> Match[match-service :3002 interno]
  Gateway --> Tournament[tournament-service :3003 interno]
  Catalog --> DB[(mysql :3306)]
  Match --> DB
  Tournament --> DB
  Edge --> MQTT[mosquitto :1883]
  Match --> MQTT
```